

.NET USER GROUP KARLSRUHE · 24 SEPTEMBER 2026

Playwright

Beyond the Happy Path

E2E tests, accessibility and MCP in practice. Sven Hennessen · vensas GmbH

About me



Sven Hennesen

vensas GmbH

- **Almost 20 years** of software development and architecture
- **Co-founder and co-managing director** of vensas GmbH, founded in 2023
- Consulting and development of **business software** for medium-sized companies
- Away from the computer — **hiking, cycling** or the **racing simulator**

The demo app: Deploy or Die

A deployment game. Each deployment has a 50% success rate. After a deployment you must wait 30 seconds.

Frontend	React 19, PrimeReact, Rspack — port 3000
Backend	ASP.NET Core minimal API, .NET 10 — POST /api/deploy
Database	PostgreSQL 16, started by Docker Compose
Tests	TypeScript with Playwright Test, C# with xUnit
Playwright	1.63.0 for TypeScript, 1.62.0 for .NET — the bindings follow Node, one version behind

Demo

`pnpm run dev` — deploy once, then watch the cooldown block the next deployment.

UI mode

```
pnpm run test:ui
```

What you get

- Watch mode — the test runs again after each file change
- Time travel — go back to each step and see the DOM
- Pick locator — click an element to get the locator
- Network, console and source in one window

Demo

pnpm run test:ui — pick a locator with the mouse, then change the test and let it run again.

FROM A REAL PROJECT

Keep the tests in their own folder

playwright.config.ts

```
// We did this. Playwright then read all of src/ at each start.  
testDir: './src',  
  
// Keep the default. The tests have their own folder.  
testDir: './tests',
```

We made this change in one project. Then we made it again in the other direction.

What it cost us

CI The test job became slow. Then it stopped with a timeout.

UI mode The window needed some minutes to start, but not each time.

Walkthrough

Keep test code separated from source code.

Traces

playwright.config.ts

```
use: {
  trace: {
    mode: 'on',
    snapshots: { dom: true, aria: true, screen: true },
  },
  screenshot: 'only-on-failure',
  video: 'retain-on-failure',
}
```

New in 1.63: the trace records the accessibility tree and the screen, not only the DOM.

A trace contains

- A DOM snapshot for each step, plus the ARIA tree and the screen
- All network requests and answers
- The console output and the source code

Demo

`npx playwright show-trace test-results/.../trace.zip` — go back in time and change to the ARIA tab.

FROM A REAL PROJECT

Make the trace available in CI

A test can fail in CI and pass on your machine. Without the artifacts you can only guess. Thus the pipeline must keep them.

`.github/workflows/e2e.yml`

```
- name: Upload the report
  uses: actions/upload-artifact@v7
  if: always()
  with:
    name: playwright-report
    path: playwright-report/
    retention-days: 7

- name: Upload traces and videos
  uses: actions/upload-artifact@v7
  if: failure()
  with:
    name: playwright-traces
    path: test-results/
    retention-days: 7
```

The report goes up after each run. The trace and the video go up only after a failure.

Walkthrough

Add the two steps to your workflow. The first one runs always. The second one runs only after a failure.

Keep the test pyramid

Codegen, UI mode and the agents make an E2E test look cheap. It is not. An E2E test needs more time to write and to keep than a unit test.

From a team that I mentored

The team wrote mostly Playwright tests. React Testing Library was sufficient for 80% of these scenarios. The suite became very large, and then the CI runners became unreliable.

What I take from this

Many

Unit and component tests. They are fast and cheap.

Few

E2E tests. Use them for the important paths.

An unreliable E2E test is worse than no E2E test.

Walkthrough

Count the E2E tests in your project. Then ask which of them a unit test or a component test can do.

Mock the API

tests/deploy-failure.spec.ts

```
// Set the route before you go to the page.
await page.route('**/api/deploy', async (route) => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({
      success: false,
      message: '💥 BOOM! The database just went on vacation.',
      deployedBy: 'Unlucky Developer',
    }),
  });
});

await page.goto('/');
```

You control the answer. Thus you can test the error path, which occurs only 50% of the time.

Demo

pnpm run test:mocked — 17 tests in about 10 seconds, with no backend and no database.

The real backend

tests/testcontainers.ts

```
const composeEnvironment = await new DockerComposeEnvironment(
  composeFilePath,
  'docker-compose.integration.yml',
).up();

const backend = composeEnvironment.getContainer('backend-1');
process.env.API_URL = `http://localhost:${backend.getMappedPort(5000)}`;
```

Docker Compose is more stable than a container that you build in the test code.

Mock or real

Mock

Fast and stable. Blind to the backend.

Real

Finds integration faults. Slow.

Use mocks for the many tests. Use the real backend for the few important paths.

Demo

pnpm run test:integration — the containers start, then 4 tests run against the real API.

Start the dependencies with Compose

First I started each container from the test code. It was flaky — on my machine and in CI.

What broke

- The start order and the readiness check
- The network between the containers and the port mapping
- Containers that stayed after the run

tests/docker-compose.integration.yml

```
postgres:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
backend:
  depends_on:
    postgres:
      condition: service_healthy
```

depends_on with a healthcheck replaces the wait code in the test.

Walkthrough

Put the healthcheck and depends_on in the Compose file. Then the test code needs no wait code.

FROM A REAL PROJECT

Mock or real — how I choose

The two approaches are not in competition. Both are correct, and both add value. I choose with the complexity of the project.

Three questions before I decide

Services How many services does the scenario touch?

Contract How complex is the API contract?

Test How complex is the test itself?

I answer these questions again for each project. Your environment and your test plan give the answer, not a rule.

Walkthrough

Answer the three questions for your own project. The demo app has one service and a small contract.

FROM A REAL PROJECT

The E2E tests are not only a frontend job

Playwright tests a user interface. Thus teams put the tests in the frontend pipeline only. But a test against a real backend image also tests the backend.

What happened to a team that I mentored

The team released an API that was not compatible. Nobody ran the E2E tests before the release.

What I recommend

- Run the tests after each frontend build
- Run them after each backend build too
- Or run them nightly — your test plan decides

Walkthrough

Find out where your E2E tests run today. Then add the backend build as a trigger, or run them nightly.

Accessibility as a gate

tests/accessibility.spec.ts

```
const results = await new AxeBuilder({ page })
  .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa', 'wcag22aa'])
  .analyze();

await testInfo.attach('axe-results.json', {
  body: JSON.stringify(results.violations, null, 2),
  contentType: 'application/json',
});

expect(results.violations.map((v) => ({ rule: v.id, impact: v.impact }))).toEqual([]);
```

The European Accessibility Act applies since June 2025. Thus this check belongs in the pipeline.

Be honest about the limit

An automatic scan finds about 30% to 50% of the WCAG problems. It does not replace a test with a screen reader.

Demo

pnpm run test:a11y fails with button-name, color-contrast and label. Set ACCESSIBLE_MODE = true in src/a11yMode.ts and it passes.

The same test in C#

DeployTests.cs

```
public class DeployTests : PageTest
{
    [Fact]
    public async Task Cooldown_BlocksTheSecondDeployment()
    {
        await Page.Clock.InstallAsync();
        await Page.GotoAsync("http://localhost:3000");

        var deploy = Page.GetByRole(AriaRole.Button,
            new() { NameRegex = new Regex("Deploy Now") });
        await deploy.ClickAsync();

        // Move the time forward instead of a wait of 30 seconds.
        await Page.Clock.FastForwardAsync("00:30");
        await Expect(deploy).ToBeEnabledAsync();
    }
}
```

The API is equal in all languages. Only the names use the .NET style. The package is Microsoft.Playwright.Xunit.v3 1.62.0 — the bindings follow Node, one version behind. NUnit and MSTest also exist.

Demo

dotnet test — 4 tests pass. Compare the C# file with the TypeScript file.

What the .NET bindings give you

The browser API is the same. The test runner is not — in .NET your runner is xUnit, NUnit or MSTest, not Playwright Test.

Works in C#

Locators	GetByRole, GetByLabel, auto-wait
Clock API	InstallAsync, FastForwardAsync
ARIA snapshots	ToMatchAriaSnapshotAsync
Mocking	RouteAsync and the API request context
Traces	Recorded, then read with show-trace
Codegen	playwright codegen --target csharp

Playwright Test only

Test locks	Use an xUnit collection instead
Isolated retries	The runner controls the retries
UI mode	No equivalent
Speedboard	Use your own test report
Test Agents	They write TypeScript

Demo

`npx playwright codegen --target csharp-xunit http://localhost:3000` — the recorder writes C#, not TypeScript.

Accessibility in C#

AccessibilityTests.cs

```
var results = await Page.RunAxe(new AxeRunOptions
{
    RunOnly = new RunOnlyOptions
    {
        Type = "tag",
        Values = ["wcag2a", "wcag2aa", "wcag21a", "wcag21aa", "wcag22aa"],
    },
});

Assert.Empty(results.Violations.Select(v => v.Id).ToArray());
```

Deque.AxeCore.Playwright 4.13.0 adds RunAxe() to Page and Locator. It uses the same axe-core engine as the TypeScript package.

Same engine, same result

Both languages report the same three rules on our form: button-name, color-contrast and label. Accessibility is not a JavaScript topic.

Demo

dotnet test — 5 tests, 1 red. The C# result names the same three rules as the TypeScript result.

The Playwright MCP server

The MCP server gives an AI agent a real browser. Since 1.62 the server is part of Playwright.

```
npx playwright mcp
```

No separate installation is necessary.

Why it is different from a screenshot agent

Input	The agent gets the accessibility tree, not pixels
Result	Fewer tokens, and the same answer each time
Effect	The agent writes role based locators, because it sees roles

Demo

Give the agent this task: explore <http://localhost:3000>. It finds the cooldown without a hint.

Where you must hold the reins

A generated test can look correct and be worthless

Typical defects: a hard coded wait, a CSS selector instead of a role, and an assertion that is always true.

```
// Generated — and wrong.  
await page.waitForTimeout(3000);  
await page.click('.p-button.p-component.deploy-button');  
await expect(page.locator('body')).toBeVisible();
```

This test passes. It tests nothing.

The rule that works for us

Agent

Explores and writes the first draft.

Human

Reviews, then puts it in the pipeline.

Demo

Find the three defects in the listing above. Then compare them with `tests/locators.spec.ts`, the reviewed version of the same test.

Lessons learned

1

Divide the tests into projects MOCKED · INTEGRATION

The mocked project needs no container. Thus a developer gets an answer in 10 seconds.

2

Use the real backend only for the important paths

Each integration test costs minutes. Four of them are enough for us.

3

Start the containers with Compose, not in the test code

A global setup with Docker Compose is much more stable than a container per test.

4

Switch on traces in the pipeline

A trace answers the question 'why did it fail in CI' without a second run.

5

Make accessibility a gate on day one

Three defects in a small form. A large application has hundreds.

That's all folks!

THANK YOU

Questions?

Demo project and slides available at <https://github.com/vensas/playwright-2026-talk>

kontakt@vensas.de

vensas.de

